

# Adding repository custom instructions for GitHub Copilot

Create a file in a repository that gives Copilot additional context for the work it does in that repository.

Visual Studio   Visual Studio Code   Web browser

---

## In this article

About repository custom instructions and prompt files for GitHub Copilot Chat

Prerequisites for repository custom instructions

Creating a repository custom instructions file

Writing effective repository custom instructions

Repository custom instructions in use

Enabling or disabling repository custom instructions

Enabling and using prompt files

---

## Note

Repository custom instructions are currently supported for Copilot Chat in Visual Studio, VS Code and on the GitHub website.

This version of this article is for using repository custom instructions in VS Code. Click the tabs above for instructions on using custom instructions in other environments.

For an overview of the methods you can use to customize GitHub Copilot Chat responses, see [About customizing GitHub Copilot Chat responses](https://docs.github.com/en/enterprise-cloud@latest/copilot/customizing-copilot/adding-repository-custom-instructions-for-github-copilot?tool=vscode).



# About repository custom instructions and prompt files for GitHub Copilot Chat

GitHub Copilot can provide chat responses that are tailored to the way your team works, the tools you use, or the specifics of your project, if you provide it with enough context to do so. Instead of repeatedly adding this contextual detail to your chat questions, you can create files in your repository that automatically add this information for you.

There are two types of files you can use to provide context and instructions to GitHub Copilot Chat in VS Code:

- **Repository custom instructions** allow you to specify repository-wide instructions and preferences, in a single file, that apply to any conversation held in the context of the repository.
- **Prompt files** (public preview) allow you to save common prompt instructions and relevant context in Markdown files ( `*.prompt.md` ) that you can then reuse in your chat prompts. Prompt files are only available in VS Code.

While custom instructions help to add codebase-wide context to each AI workflow, prompt files let you add instructions to a specific chat interaction.

## Repository custom instructions example

This example of a `.github/copilot-instructions.md` file contains three instructions that will be added to all chat questions.

```
We use Bazel for managing our Java dependencies, not Maven, so when talking about
Java packages. always give me instructions and code samples that use Bazel.
```



GitHub Docs

Version: Enterprise Cloud ▾



≡ GitHub Copilot / Customize Copilot / Repository custom instructions

## Prompt file examples

The following examples demonstrate how to use prompt files.

- `New React form.prompt.md` - contains instructions for a reusable task to generate a form using React.

Your goal is to generate a new React form component.

Ask for the form name and fields if not provided.

Requirements for the form:

- Use form design system components: `[design-system/Form.md](../docs/design-system/Form.md)`
- Use ``react-hook-form`` for form state management:
  - Always define TypeScript types for your form data
  - Prefer `*uncontrolled*` components using `register`
  - Use ``defaultValues`` to prevent unnecessary rerenders
- Use ``yup`` for validation:
  - Create reusable validation schemas in separate files
  - Use TypeScript types to ensure type safety
  - Customize UX-friendly validation rules

- API security review.prompt.md - contains reusable information about security practices for REST APIs, which can be used to do security reviews of REST APIs.

Secure REST API review:

- Ensure all endpoints are protected by authentication and authorization
- Validate all user inputs and sanitize data
- Implement rate limiting and throttling
- Implement logging and monitoring for security events
- ...

## Prerequisites for repository custom instructions

- A custom instructions file (see the instructions below).
- The **Use Instruction Files** option must be enabled in your settings. This is enabled by default. See [Enabling or disabling repository custom instructions](#) later in this article.

## Creating a repository custom instructions file

- 1 In the root of your repository, create a file named `.github/copilot-instructions.md`.

Create the `.github` directory if it does not already exist.

- 2 Add natural language instructions to the file, in Markdown format.

Whitespace between instructions is ignored, so the instructions can be written as a single paragraph, each on a new line, or separated by blank lines for legibility.

To see your instructions in action, go to <https://github.com/copilot>, attach the repository containing the instructions file, and start a conversation.

Did you successfully add a custom instructions file to your repository?

Yes

No

## Writing effective repository custom instructions

The instructions you add to the `.github/copilot-instructions.md` file should be short, self-contained statements that add context or relevant information to supplement users' chat questions.

You should also consider the size and complexity of your repository. The following types of instructions may work for a small repository with only a few contributors, but for a large and diverse repository, they may cause problems with other areas of Copilot:

- Requests to refer to external resources when formulating a response
- Instructions to answer in a particular style
- Requests to always respond with a certain level of detail

For example, the following instructions may not have the intended results:

```
Always conform to the coding styles defined in styleguide.md in repo my-org/my-repo when generating code.
```

```
Use @terminal when answering questions about Git.
```

```
Answer all questions in the style of a friendly colleague, using informal language.
```

```
Answer all questions in less than 1000 characters, and words of no more than 12 characters.
```

## Repository custom instructions in use

The instructions in the `.github/copilot-instructions.md` file are available for use by Copilot Chat as soon as you save the file. The complete set of instructions will be automatically attached to requests that you submit in either the Copilot Chat view, or in inline chat.

Custom instructions are not visible in the Chat view or inline chat, but you can verify that they are being used by Copilot by looking at the References list of a response in the Chat view. If custom instructions were added to the prompt that was sent to the model, the `.github/copilot-instructions.md` file is listed as a reference. You can click the reference to open the file.



## Enabling or disabling repository custom instructions

You can choose whether or not to have custom instructions added to your chat questions.

- 1 Open the Setting editor by using the keyboard shortcut `Command + ,` (Mac) / `Ctrl + ,` (Linux/Windows).
- 2 Type `instruction file` in the search box.
- 3 Select or clear the checkbox under **Code Generation: Use Instruction Files**.

## Enabling and using prompt files

### Note

Prompt files are public preview and subject to change.

Prompt files let you build and share reusable prompt instructions with additional context. A prompt file is a Markdown file, stored in your workspace, that mimics the existing format of writing prompts in Copilot Chat (for example, `Rewrite #file:x.ts`). You can have multiple prompt files in your workspace, each of which defines a prompt for a different purpose.

## Enabling prompt files

To enable prompt files, configure the workspace settings.

- 1 Open the command palette by pressing `Ctrl + Shift + P` (Windows/Linux) / `Command + Shift + P` (Mac).
- 2 Type "Open Workspace Settings (JSON)" and select the option that's displayed.
- 3 In the `settings.json` file, add `"chat.promptFiles": true` to enable the `.github/prompts` folder as the location for prompt files. This folder will be created if it does not already exist.

## Creating prompt files

- 1 Open the command palette by pressing `Ctrl + Shift + P` (Windows/Linux) / `Command + Shift + P` (Mac).
- 2 Type "prompt" and select **Chat: Create Prompt**.
- 3 Enter a name for the prompt file, excluding the `.prompt.md` file name extension. The name can contain alphanumeric characters and spaces and should describe the purpose of the prompt information the file will contain.
- 4 Write the prompt instructions, using Markdown formatting.

You can reference other files in the workspace by using Markdown links—for example, `[index](../../web/index.ts)` —or by using the `#file:../../web/index.ts` syntax. Paths are relative to the prompt file. Referencing other files allows you to provide additional context, such as API specifications or product documentation.

## Using prompt files

- 1 At the bottom of the Copilot Chat view, click the **Attach context** icon (.
- 2 In the dropdown menu, click **Prompt...** and choose the prompt file you want to use.
- 3 Optionally, attach additional files, including prompt files, to provide more context.
- 4 Optionally, type additional information in the chat prompt box.

Whether you need to do this or not depends on the contents of the prompt you are using.

- 5 Submit the chat prompt.

For more information about prompt files, see [Custom instructions for GitHub Copilot in VS Code](#) in the Visual Studio Code documentation.

## Legal

© 2025 GitHub, Inc. [Terms](#) [Privacy](#) [Status](#) [Pricing](#) [Expert services](#) [Blog](#)